

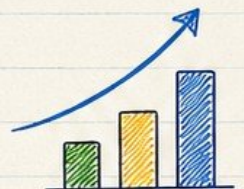
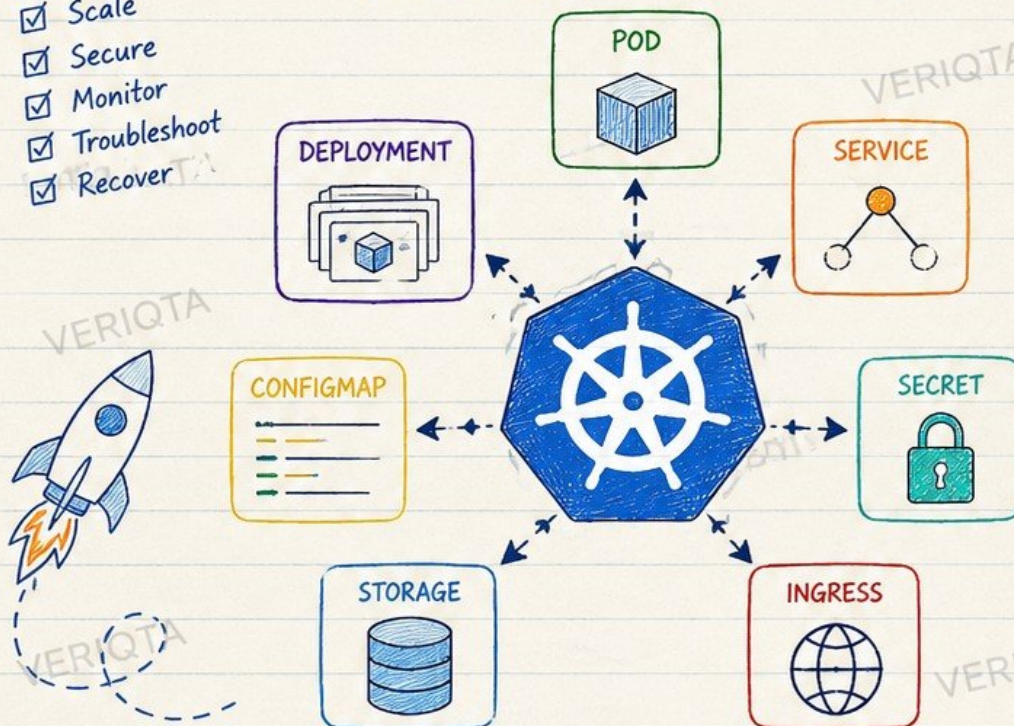


PRACTICAL KUBERNETES HANDBOOK

From Local Cluster to Production Operations

- ✓ Deploy
- ✓ Scale
- ✓ Secure
- ✓ Monitor
- ✓ Troubleshoot
- ✓ Recover

★ Learn by Doing
Build by Practicing
Master by Solving



A PRACTICAL, HANDS-ON GUIDE TO



- ✓ Build Kubernetes Clusters
- ✓ Deploy and Manage Applications
- ✓ Operate, Troubleshoot and Recover
- ✓ Design Production-Ready Systems

Hands-on
Labs
Real-world
Scenarios
Production
Ready



PAGE 1: BUILD YOUR KUBERNETES LAB



WHAT YOU WILL DO

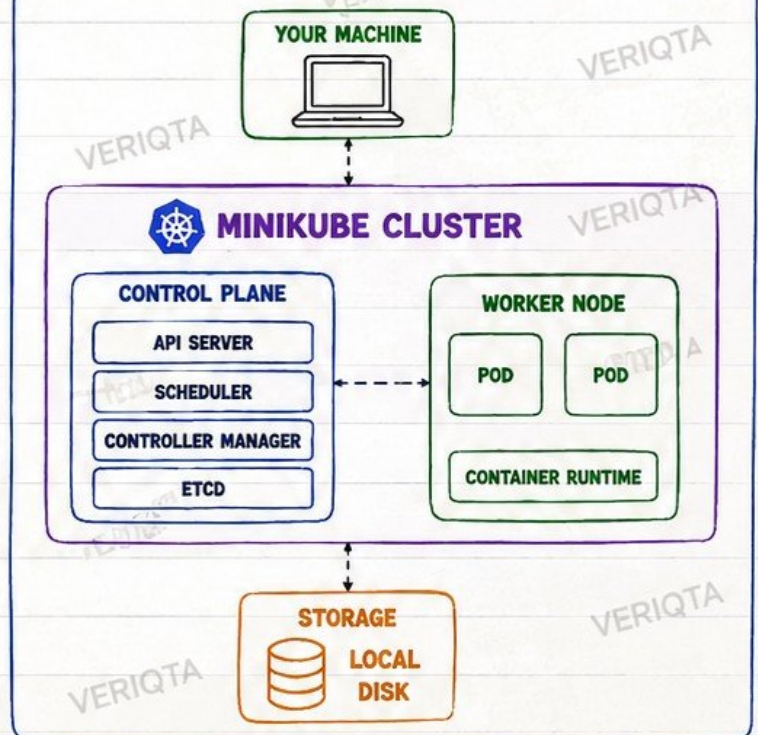
- 1 INSTALL KUBECTL
- 2 INSTALL DOCKER
- 3 INSTALL MINIKUBE
- 4 START THE CLUSTER
- 5 VERIFY CLUSTER CONNECTIVITY
- 6 INSPECT NODES
- 7 CHECK CLUSTER COMPONENTS
- 8 UNDERSTAND THE LAB ARCHITECTURE



LAB GOALS

- ✓ CREATE A WORKING KUBERNETES CLUSTER LOCALLY
- ✓ VERIFY THAT ALL COMPONENTS ARE RUNNING
- ✓ INSPECT NODES AND PODS
- ✓ UNDERSTAND HOW THE CLUSTER IS STRUCTURED

KUBERNETES LAB ARCHITECTURE



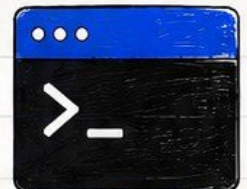
IMPORTANT

MINIKUBE RUNS A SINGLE-NODE KUBERNETES CLUSTER INSIDE A VIRTUAL MACHINE. PERFECT FOR LEARNING AND PRACTICE.



ESSENTIAL COMMANDS

- | | |
|--|--|
| <code>minikube start</code> | - START MINIKUBE CLUSTER |
| <code>minikube status</code> | - CHECK MINIKUBE STATUS |
| <code>kubectl cluster-info</code> | - SHOW CLUSTER INFORMATION |
| <code>kubectl get nodes</code> | - LIST NODES IN THE CLUSTER |
| <code>kubectl get nodes -o wide</code> | - LIST NODES WITH DETAILED INFORMATION |
| <code>kubectl get pods -A</code> | - LIST ALL PODS IN ALL NAMESPACES |



PRACTICAL TASK: BUILD A WORKING KUBERNETES CLUSTER AND VERIFY THAT IT IS HEALTHY.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



PAGE 2: DEPLOY YOUR FIRST APPLICATION



WHAT YOU WILL DO

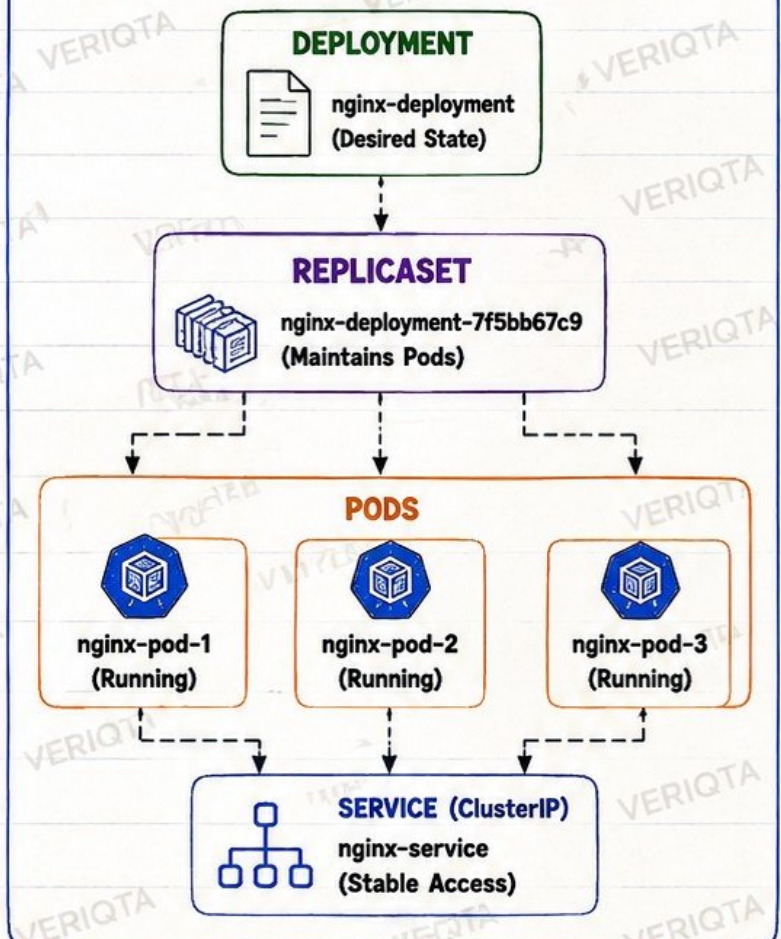
- 1 CREATE A DEPLOYMENT
- 2 DEPLOY NGINX
- 3 INSPECT THE DEPLOYMENT
- 4 INSPECT REPLICASETS
- 5 INSPECT PODS
- 6 VIEW POD DETAILS
- 7 ACCESS THE APPLICATION
- 8 DELETE AND RECREATE RESOURCES



WHAT IS HAPPENING?

- ✓ YOU CREATE A DEPLOYMENT.
- ✓ KUBERNETES CREATES A REPLICASET.
- ✓ THE REPLICASET CREATES AND MAINTAINS THE PODS.
- ✓ THE SERVICE PROVIDES STABLE ACCESS TO THE PODS.

KUBERNETES WORKLOAD RELATIONSHIP



ESSENTIAL COMMANDS

```

kubect1 create deployment nginx --image=nginx
kubect1 get deployments
kubect1 get rs
kubect1 get pods
kubect1 describe deployment nginx
kubect1 describe pod <pod-name>
  
```

- DEPLOY NGINX
- LIST DEPLOYMENTS
- LIST REPLICASETS
- LIST PODS
- INSPECT DEPLOYMENT DETAILS
- INSPECT POD DETAILS



PRACTICAL TASK

DEPLOY NGINX AND TRACE HOW DEPLOYMENT, REPLICASET, AND POD ARE RELATED IN KUBERNETES.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



PAGE 3: WORK WITH KUBERNETES YAML



WHAT YOU WILL DO

BUILD A DEPLOYMENT MANIFEST FROM SCRATCH.



PRACTICE

- ☒ APIVERSION
- ☒ KIND
- ☒ METADATA
- ☒ SPEC
- ☒ LABELS
- ☒ SELECTORS
- ☒ POD TEMPLATE
- ☒ CONTAINER SPECIFICATION
- ☒ IMAGE CONFIGURATION
- ☒ PORTS

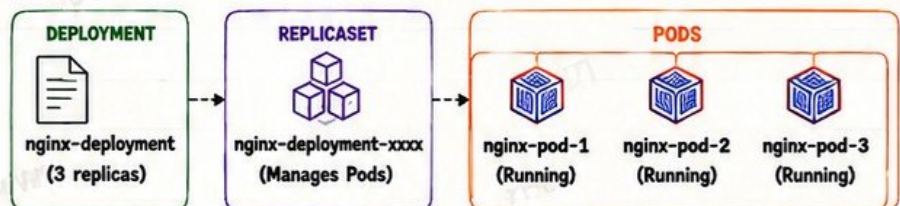


YAML BEST PRACTICES

- ✓ USE SPACES, NOT TABS
- ✓ KEEP INDENTATION CONSISTENT
- ✓ USE DESCRIPTIVE NAMES
- ✓ KEEP MANIFESTS IN VERSION CONTROL
- ✓ APPLY CHANGES DECLARATIVELY

SAMPLE DEPLOYMENT.YAML

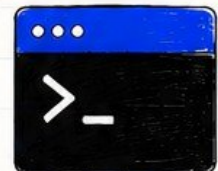
```
apiVersion: apps/v1      ← API version for Deployment
kind: Deployment         ← Resource type
metadata:
  name: nginx-deployment ← Name of the Deployment
  labels:
    app: nginx           ← Label for the Deployment
spec:
  replicas: 3            ← Number of Pod replicas
  selector:
    matchLabels:
      app: nginx         ← Selects Pods with label app=nginx
  template:
    metadata:
      labels:             ← Pod template labels
      app: nginx
    spec:
      containers:
        - name: nginx     ← Container name
          image: nginx:latest ← Container image
          ports:
            - containerPort: 80 ← Port exposed by container
```



ESSENTIAL COMMANDS

```
kubectl apply -f deployment.yaml
kubectl get deployment
kubectl get pods
kubectl get deployment -o yaml
kubectl delete -f deployment.yaml
```

- APPLY THE DEPLOYMENT MANIFEST
- LIST DEPLOYMENTS
- LIST PODS
- VIEW DEPLOYMENT IN YAML FORMAT
- DELETE THE DEPLOYMENT



PRACTICAL TASK

STOP RELYING ON IMPERATIVE COMMANDS AND DEPLOY THE APPLICATION DECLARATIVELY.

IMPERATIVE

```
kubectl create ...
kubectl expose ...
```



DECLARATIVE

```
deployment.yaml
```



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



PAGE 4: EXPOSE APPLICATIONS WITH SERVICES



WHAT YOU WILL DO

- ✓ UNDERSTAND SERVICE TYPES
- ✓ CREATE A SERVICE
- ✓ UNDERSTAND SELECTORS
- ✓ UNDERSTAND ENDPOINTS
- ✓ DISCOVER SERVICES
- ✓ TEST CONNECTIVITY



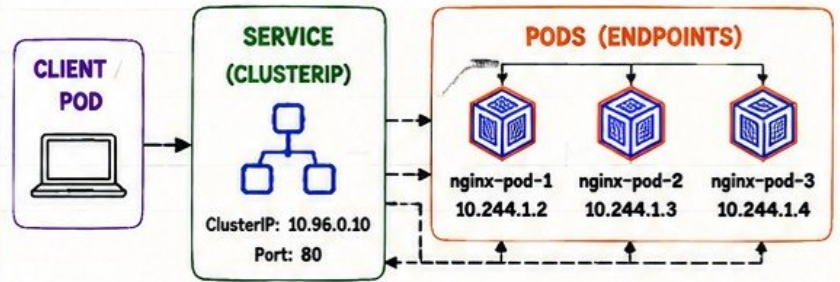
KEY CONCEPTS

- CLUSTERIP: INTERNAL ACCESS
- NODEPORT: EXPOSE ON A NODE PORT
- SELECTORS: CONNECT SERVICE TO THE CORRECT PODS
- TARGET PORT: PORT ON THE POD
- ENDPOINTS: POD IPs BEHIND SERVICE
- SERVICE DISCOVERY: DNS BASED

BUILD: service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  labels:
    app: nginx
spec:
  type: ClusterIP      # Change to NodePort if needed
  selector:
    app: nginx        # Selects pods with label app=nginx
  ports:
    - port: 80        # Service port
      targetPort: 80  # Container port on the pod
      protocol: TCP
```

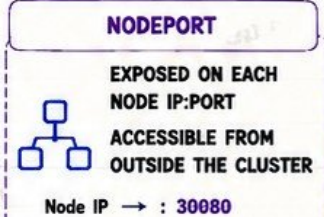
SERVICE ARCHITECTURE (CLUSTERIP EXAMPLE)



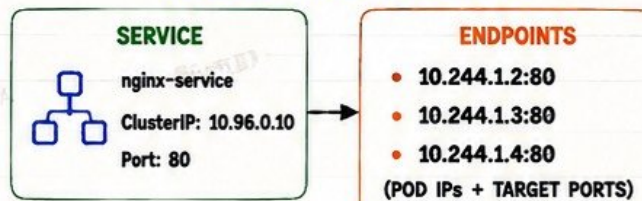
FLOW

CLIENT → SERVICE (CLUSTERIP) → ENDPOINTS (PODS)

SERVICE TYPES



SERVICE AND ENDPOINTS



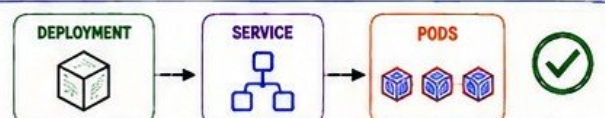
ESSENTIAL COMMANDS

- ```
kubectl expose deployment nginx --port=80 --type=ClusterIP
```
- CREATE CLUSTERIP SERVICE FROM DEPLOYMENT
- ```
kubectl get svc
```
- LIST ALL SERVICES
- ```
kubectl describe svc nginx
```
- SHOW SERVICE DETAILS
- ```
kubectl get endpoints
```
- LIST ENDPOINTS (PODS BEHIND SERVICE)
- ```
kubectl port-forward svc/nginx 8080:80
```
- FORWARD LOCAL PORT 8080 TO SERVICE PORT 80
- ```
curl localhost:8080
```
- TEST CONNECTIVITY TO THE APPLICATION



PRACTICAL TASK

EXPOSE THE APPLICATION AND VERIFY THAT SERVICE TRAFFIC REACHES THE CORRECT PODS.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



PAGE 5: LABELS, SELECTORS, AND NAMESPACES



WHAT YOU WILL DO

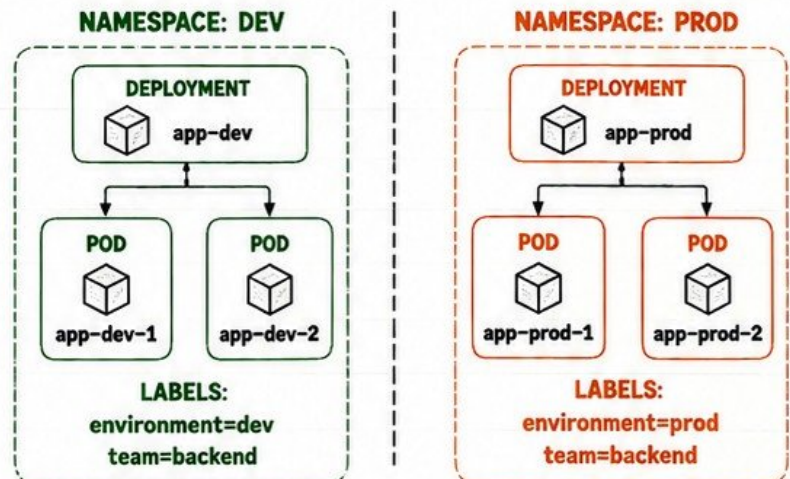
- 1 CREATE NAMESPACES
- 2 DEPLOY WORKLOADS INTO NAMESPACES
- 3 ADD LABELS
- 4 MODIFY LABELS
- 5 QUERY RESOURCES BY LABEL
- 6 TEST SELECTORS
- 7 ORGANIZE ENVIRONMENTS



KEY CONCEPTS

- NAMESPACES: ISOLATE RESOURCES
- LABELS: KEY-VALUE PAIRS ATTACHED TO OBJECTS
- SELECTORS: MATCH LABELS TO FIND RESOURCES
- USE LABELS TO ORGANIZE, QUERY, AND MANAGE RESOURCES
- KEEP DEV AND PROD ENVIRONMENTS SEPARATE

ORGANIZE YOUR CLUSTER WITH NAMESPACES



NAMESPACES PROVIDE ISOLATION AND ORGANIZATION FOR DIFFERENT ENVIRONMENTS.

LABELS AND SELECTORS



SELECTORS FIND RESOURCES THAT MATCH THE LABELS.



ESSENTIAL COMMANDS

```
kubectl create namespace dev
```

```
kubectl create namespace prod
```

```
kubectl get namespaces
```

```
kubectl get pods -n dev
```

```
kubectl label pod <pod-name> environment=dev
```

```
kubectl get pods -l environment=dev
```

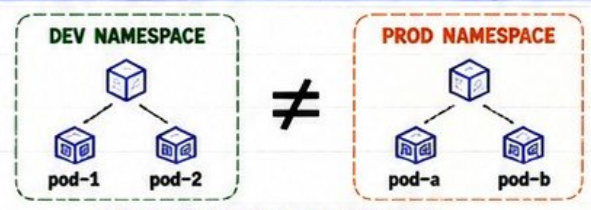
```
kubectl get all -n dev
```

- CREATE A NAMESPACE: dev
- CREATE A NAMESPACE: prod
- LIST ALL NAMESPACES
- LIST PODS IN THE dev NAMESPACE
- ADD/UPDATE LABEL ON A POD
- LIST PODS WITH LABEL environment=dev
- LIST ALL RESOURCES IN dev NAMESPACE



PRACTICAL TASK

- ✓ CREATE SEPARATE DEV AND PROD NAMESPACES
- ✓ DEPLOY APPLICATIONS INTO EACH NAMESPACE
- ✓ APPLY LABELS TO PODS
- ✓ USE SELECTORS TO QUERY RESOURCES
- ✓ VERIFY THAT RESOURCES ARE ISOLATED BY NAMESPACE



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



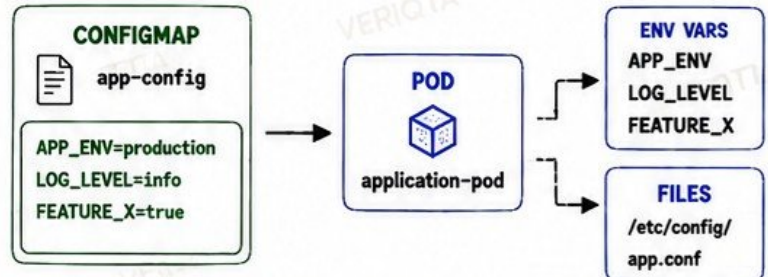
PAGE 6: CONFIGURE APPLICATIONS WITH CONFIGMAPS



WHAT YOU WILL DO

- 1 CREATE CONFIGMAPS
- 2 CONFIGURATION FROM LITERALS
- 3 CONFIGURATION FROM FILES
- 4 USE AS ENVIRONMENT VARIABLES
- 5 MOUNT CONFIGURATION AS FILES
- 6 UPDATE APPLICATION CONFIGURATION WITHOUT REBUILDING IMAGE

HOW CONFIGMAPS WORK



CONFIGMAPS STORE NON-SENSITIVE CONFIGURATION DATA AS KEY-VALUE PAIRS.

CREATE CONFIGMAPS: EXAMPLES

FROM LITERAL VALUES

```
>_
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_X=true
```

RESULT:

KEY	VALUE
APP_ENV	production
LOG_LEVEL	info
FEATURE_X	true

FROM FILE (APP.CONF)

```
APP_ENV=production
LOG_LEVEL=debug
FEATURE_X=true
MAX_CONNECTIONS=200
```

```
>_
kubectl create configmap
app-config \
  --from-file=app.conf
```

VIEW CONFIGMAP DATA

```
>_ kubectl get configmap app-config -o yaml
```

WAYS TO USE CONFIGMAPS IN PODS

AS ENVIRONMENT VARIABLES

```
env:
- name: APP_ENV
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: APP_ENV
```

2. AS VOLUME (FILES)

```
volumeMounts:
- name: config-volume
  mountPath: /etc/config

volumes:
- name: config-volume
  configMap:
    name: app-config
```



BENEFITS

- SEPARATE CONFIGURATION FROM IMAGE
- ✓ UPDATE CONFIGURATION WITHOUT REBUILDING
- ✓ EASY TO MANAGE AND REUSE
- ✓ SAFE FOR NON-SENSITIVE DATA



ESSENTIAL COMMANDS

```
>_ kubectl create configmap app-config --from-literal=APP_ENV=production
>_ kubectl get configmaps
>_ kubectl describe configmap app-config
>_ kubectl get configmap app-config -o yaml
```

- CREATE CONFIGMAP FROM LITERAL VALUE
- LIST ALL CONFIGMAPS
- SHOW DETAILED INFORMATION
- VIEW CONFIGMAP IN YAML FORMAT



PRACTICAL TASK

CHANGE APPLICATION BEHAVIOR WITHOUT REBUILDING THE CONTAINER IMAGE.

UPDATE CONFIGMAP



APPLICATION READS NEW CONFIGURATION



BEHAVIOR CHANGES



EXAMPLE: CHANGE LOG_LEVEL FROM info TO debug AND OBSERVE THE APPLICATION.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



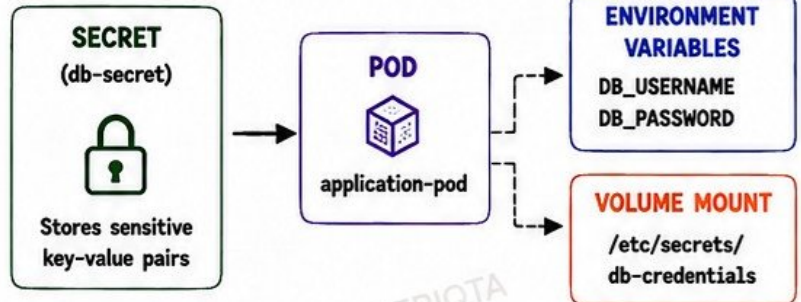
PAGE 7: MANAGE SENSITIVE CONFIGURATION WITH SECRETS



WHAT YOU WILL DO

- 1 CREATE SECRETS
- 2 GENERIC SECRETS
- 3 USE AS ENVIRONMENT VARIABLES
- 4 MOUNT SECRETS AS VOLUMES
- 5 INSPECT SECRET METADATA
- 6 UNDERSTAND BASE64 ENCODING
- 7 AVOID EXPOSING CREDENTIALS

HOW SECRETS WORK



SECRETS KEEP SENSITIVE INFORMATION OUT OF IMAGES AND SOURCE CODE.

TYPES OF SECRETS



GENERIC SECRET

Stores arbitrary key-value pairs (username, password, token, etc.)

• OTHER SECRET TYPES

- TLS SECRET (CERTIFICATES)
- DOCKER CONFIG SECRET
- SERVICE ACCOUNT TOKEN SECRET

HOW SECRETS ARE STORED

YOUR DATA

username: admin
password: mypass123

KUBERNETES STORES (BASE64 ENCODED)

username: YWRtaW4=
password: bXlyYXNzMTIz



BASE64 IS NOT ENCRYPTION.
IT IS ONLY ENCODING.
USE RBAC TO PROTECT SECRETS.



BEST PRACTICES

- ✓ NEVER STORE SECRETS IN IMAGES OR CODE
- ✓ USE LEAST PRIVILEGE (RBAC)
- ✓ LIMIT ACCESS TO SECRETS
- ✓ USE NAMESPACES TO ISOLATE
- ✓ ROTATE SECRETS REGULARLY
- ✓ AUDIT SECRET ACCESS
- ✓ DO NOT LOG SECRETS

EXAMPLE: GENERIC SECRET

```
>_ kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=<password>
```

KEY	VALUE (BASE64 ENCODED)
username	YWRtaW4=
password	PD4cGFzc3dvcmQ8

VALUES ARE STORED IN BASE64 FORMAT INSIDE THE CLUSTER.



ESSENTIAL COMMANDS

```
kubectl create secret generic db-secret --from-literal=username=admin --from-literal=password=<password>
```

```
kubectl get secrets
```

```
kubectl describe secret db-secret
```

- CREATE GENERIC SECRET WITH LITERAL VALUES
- LIST ALL SECRETS IN CURRENT NAMESPACE
- SHOW DETAILS AND METADATA OF THE SECRET



PRACTICAL TASK

INJECT DATABASE CREDENTIALS INTO AN APPLICATION WITHOUT HARDCODING THEM IN THE CONTAINER IMAGE.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



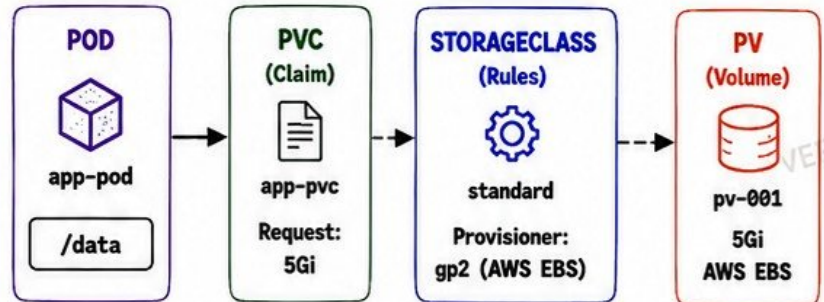
PAGE 8: PERSISTENT STORAGE



WHAT YOU WILL DO

- 1 PERSISTENTVOLUME (PV)
- 2 PERSISTENTVOLUMECLAIM (PVC)
- 3 STORAGECLASS
- 4 DYNAMIC PROVISIONING
- 5 VOLUME MOUNTS
- 6 DATA PERSISTENCE
- 7 WRITE AND TEST PERSISTENCE

KUBERNETES STORAGE ARCHITECTURE



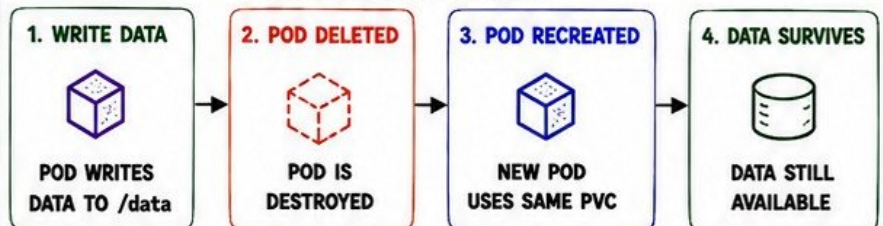
PVC REQUESTS STORAGE USING A STORAGECLASS.
KUBERNETES PROVISIONS A PV AND BINDS IT TO THE PVC.



KEY CONCEPTS

- PV: CLUSTER STORAGE RESOURCE
- PVC: STORAGE REQUEST BY USERS
- STORAGECLASS: DEFINES HOW STORAGE IS PROVISIONED
- DYNAMIC PROVISIONING: STORAGE CREATED ON DEMAND
- PODS MOUNT PVCs AS VOLUMES
- DATA REMAINS AFTER POD RECREATION

PERSISTENCE LIFECYCLE



TOP BENEFITS

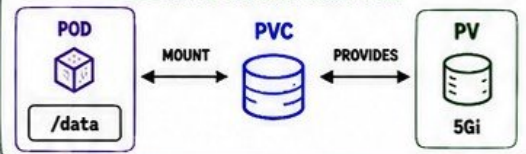
- ✓ DATA SURVIVES POD RESTARTS AND FAILURES
- ✓ SUPPORTS STATEFUL APPLICATIONS
- ✓ ABSTRACTS INFRASTRUCTURE DETAILS
- ✓ WORKS WITH DYNAMIC PROVISIONING



BUILD FILES

- pvc.yaml
- deployment.yaml

VOLUME MOUNT EXAMPLE



SUPPORTED ACCESS MODES

- ReadWriteOnce (RWO)
- ReadOnlyMany (ROX)
- ReadWriteMany (RWX)

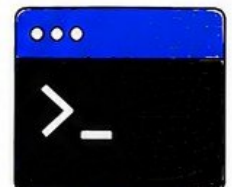


ESSENTIAL COMMANDS

- ```

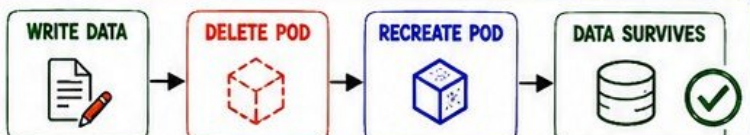
kubect1 get pv
kubect1 get pvc
kubect1 get storageclass
kubect1 describe pvc <pvc-name>
kubect1 get pods

```
- LIST ALL PERSISTENTVOLUMES
  - LIST ALL PERSISTENTVOLUMECLAIMS
  - LIST ALL STORAGECLASSES
  - SHOW DETAILS OF A SPECIFIC PVC
  - LIST ALL PODS



## PRACTICAL TASK

WRITE DATA INSIDE A POD, DESTROY THE POD,  
RECREATE IT, AND VERIFY THAT THE DATA SURVIVES.



YouTube  
@veriqta



GitHub  
@veriqta



LinkedIn  
@veriqta



X  
@veriqta



Instagram  
@veriqta



Telegram  
@veriqta



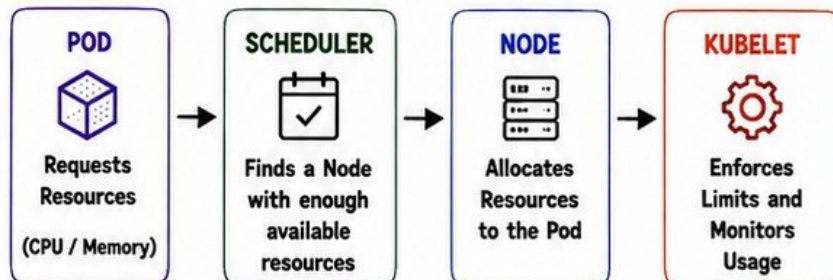
# PAGE 9: RESOURCE REQUESTS AND LIMITS



## WHAT YOU WILL DO

- 1 CPU REQUESTS
- 2 MEMORY REQUESTS
- 3 CPU LIMITS
- 4 MEMORY LIMITS
- 5 SCHEDULER DECISIONS
- 6 RESOURCE CONSUMPTION
- 7 OOMKILLED
- 8 CPU THROTTLING

## HOW RESOURCES WORK IN KUBERNETES



- REQUESTS = MINIMUM RESOURCES GUARANTEED TO THE POD
- LIMITS = MAXIMUM RESOURCES THE POD CAN USE

## RESOURCE TYPES



CPU  
(Millicores)



MEMORY  
(MiB / GiB)

## EXAMPLE: RESOURCE CONFIGURATION

```
resources:
 requests:
 cpu: "250m"
 memory: "256Mi"
 limits:
 cpu: "500m"
 memory: "512Mi"
```

- REQUESTS:
  - CPU: 250 millicores (0.25 CPU)
  - MEMORY: 256 MiB
- LIMITS:
  - CPU: 500 millicores (0.5 CPU)
  - MEMORY: 512 MiB

## RESOURCE ISSUES

- ⚠️ OOMKILLED (OUT OF MEMORY)  
CONTAINER EXCEEDS MEMORY LIMIT
- ⚠️ CPU THROTTLING  
CONTAINER EXCEEDS CPU LIMIT AND IS THROTTLED
- ⚠️ PENDING POD  
INSUFFICIENT RESOURCES ON ALL NODES

## POD LIFECYCLE WITH RESOURCE LIMITS (OOM EXAMPLE)

### 1. POD RUNNING



Using memory normally

### 2. MEMORY USAGE INCREASES



Usage exceeds limit

### 3. OOMKILLED



Container is killed by kubelet

### 4. POD RESTARTS



Pod restarts (per restartPolicy)

### 5. INVESTIGATE



Check events and logs



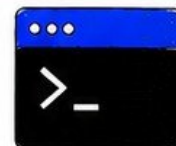
## ESSENTIAL COMMANDS

```
kubectl top nodes
```

```
kubectl top pods
```

```
kubectl describe pod <pod-name>
```

- SHOW CPU AND MEMORY USAGE FOR NODES
- SHOW CPU AND MEMORY USAGE FOR PODS
- SHOW DETAILED POD INFORMATION (EVENTS, LIMITS, STATUS)



## PRACTICAL TASK

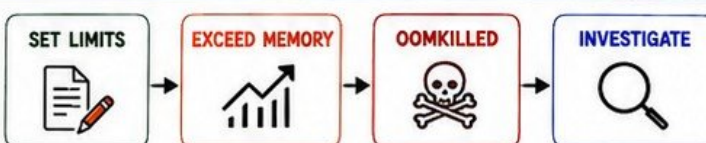
SET RESOURCE LIMITS, INTENTIONALLY EXCEED MEMORY LIMITS, AND INVESTIGATE AN OOMKILLED CONTAINER.

✓ SET MEMORY LIMIT (e.g., 256Mi)

✓ GENERATE MEMORY LOAD

✓ OBSERVE OOMKILLED

✓ INVESTIGATE WITH COMMANDS



YouTube  
@veriqta



GitHub  
@veriqta



LinkedIn  
@veriqta



X  
@veriqta



Instagram  
@veriqta



Telegram  
@veriqta



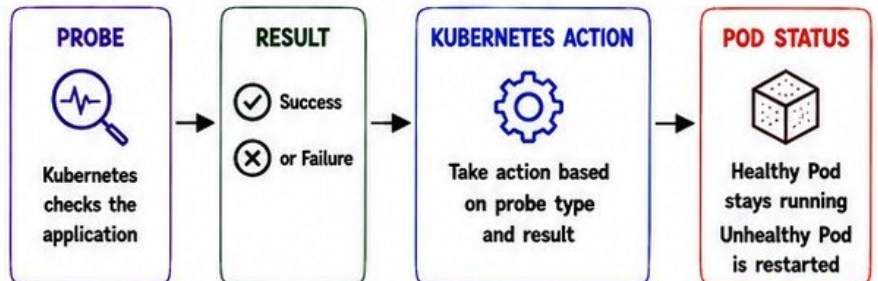
# PAGE 10: HEALTH CHECKS AND SELF-HEALING



## WHAT YOU WILL DO

- 1 LIVENESS PROBES
- 2 READINESS PROBES
- 3 STARTUP PROBES
- 4 HTTP PROBES
- 5 TCP PROBES
- 6 COMMAND PROBES
- 7 FAILED HEALTH CHECKS
- 8 POD RESTART BEHAVIOR

## HOW HEALTH CHECKS WORK



PROBES HELP KUBERNETES KNOW WHEN A CONTAINER IS HEALTHY, UNHEALTHY, OR READY TO RECEIVE TRAFFIC.

## TYPES OF PROBES

### LIVENESS PROBE



Checks if the application is alive.  
If it fails, Kubernetes restarts the container.

### READINESS PROBE



Checks if the application is ready to serve traffic.  
If it fails, Pod is removed from Service endpoints.

### STARTUP PROBE



Used for slow-starting applications.  
Disables liveness and readiness until success.

### PROBE TYPES

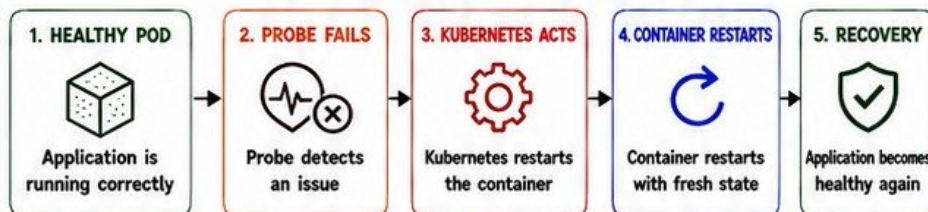
- HTTP PROBE  
Checks an HTTP endpoint
- TCP PROBE  
Checks a TCP port
- COMMAND PROBE  
Executes a command inside the container

### FAILED PROBES



Repeated failures trigger actions based on probe type and settings.  
Kubernetes keeps your application available.

## PROBE BEHAVIOR AND SELF-HEALING



SELF-HEALING KEEPS YOUR APPLICATION AVAILABLE WITHOUT MANUAL INTERVENTION.

## COMMON PROBE SETTINGS

- ☒ initialDelaySeconds  
Delay before first probe
- ☒ periodSeconds  
How often to probe
- ☒ timeoutSeconds  
How long to wait for response
- ☒ failureThreshold  
Failures before action
- ☒ successThreshold  
Successes before marking healthy



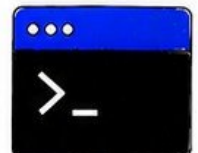
## ESSENTIAL COMMANDS

```
kubectl describe pod <pod-name>
```

```
kubectl get pods -w
```

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

- SHOW DETAILED INFORMATION ABOUT A POD (PROBES, EVENTS, RESTARTS, STATUS)
- WATCH POD STATUS IN REAL TIME
- LIST EVENTS IN CHRONOLOGICAL ORDER



## PRACTICAL TASK

CONFIGURE A FAILING HEALTH CHECK, OBSERVE KUBERNETES BEHAVIOR, FIX IT, AND VERIFY RECOVERY.



✓ UNDERSTAND PROBE DEFINITIONS

✓ OBSERVE FAILURE AND RESTARTS

✓ CORRECT CONFIGURATION

✓ ENSURE APPLICATION RECOVERS



YouTube  
@veriqta



GitHub  
@veriqta



LinkedIn  
@veriqta



X  
@veriqta



Instagram  
@veriqta



Telegram  
@veriqta



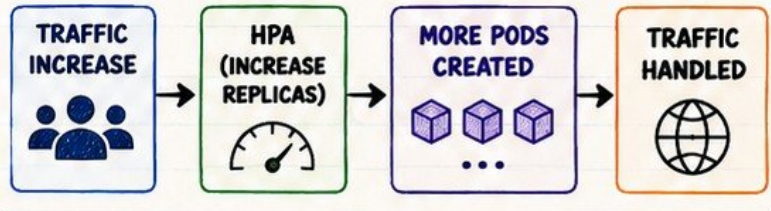
# 11. SCALING APPLICATIONS



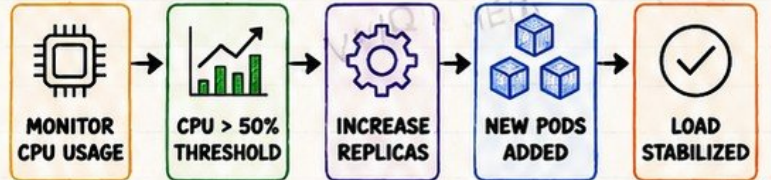
## KEY TOPICS

- MANUAL SCALING
- REPLICA MANAGEMENT
- HORIZONTAL POD AUTOSCALER
- CPU-BASED SCALING
- GENERATE TEST TRAFFIC
- OBSERVE SCALING
- SCALE DOWN

## HOW SCALING WORKS



## HPA CPU-BASED SCALING FLOW



## COMMANDS

- 1 `kubectl scale deployment web --replicas=5`
- 2 `kubectl get pods`
- 3 `kubectl autoscale deployment web --cpu-percent=50 --min=2 --max=10`
- 4 `kubectl get hpa`
- 5 `kubectl describe hpa`
- 6 `kubectl top pods`

## EXAMPLE HPA CONFIGURATION

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: web-hpa
spec:
 scaleTargetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: web
 minReplicas: 2
 maxReplicas: 10
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 50
```

HPA KEEPS YOUR APP STABLE BY ADDING PODS WHEN CPU USAGE GOES ABOVE 50%.

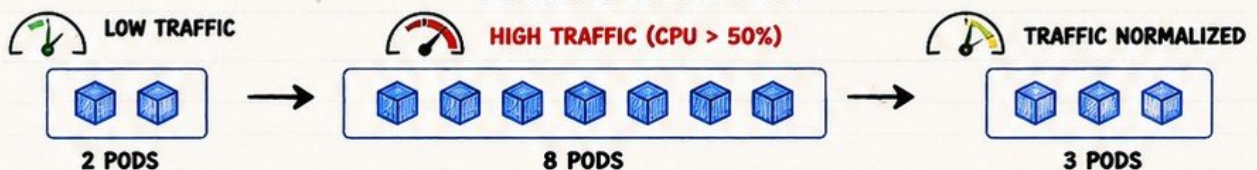


## PRACTICAL TASK

GENERATE LOAD AND WATCH

- KUBERNETES INCREASE AND DECREASE APPLICATION REPLICAS.

## SCALING IN ACTION



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



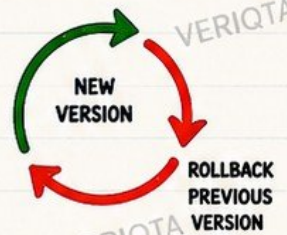
INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



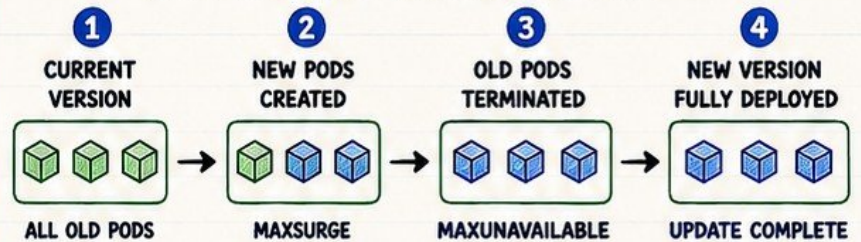
# 12. ROLLING UPDATES AND ROLLBACKS



## KEY TOPICS

- UPDATE CONTAINER IMAGES
- ROLLINGUPDATE STRATEGY
- MAXSURGE
- MAXUNAVAILABLE
- DEPLOYMENT HISTORY
- FAILED RELEASES
- ROLLBACK

## HOW ROLLING UPDATES WORK



## ROLLINGUPDATE STRATEGY



## COMMANDS

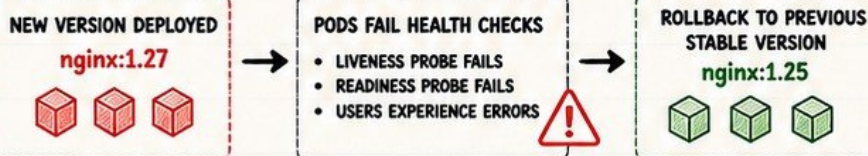
- 1 `kubectl set image deployment/web web=nginx:1.27`
- 2 `kubectl rollout status deployment/web`
- 3 `kubectl rollout history deployment/web`
- 4 `kubectl rollout undo deployment/web`

## DEPLOYMENT HISTORY EXAMPLE

| REVISION | CHANGE-CAUSE         | IMAGE      | STATUS     | AGE |
|----------|----------------------|------------|------------|-----|
| 3        | UPDATE TO NGINX 1.27 | nginx:1.27 | FAILED     | 2m  |
| 2        | UPDATE TO NGINX 1.25 | nginx:1.25 | DEPLOYED   | 20m |
| 1        | INITIAL DEPLOYMENT   | nginx:1.24 | SUPERSEDED | 1h  |



## FAILED RELEASE EXAMPLE

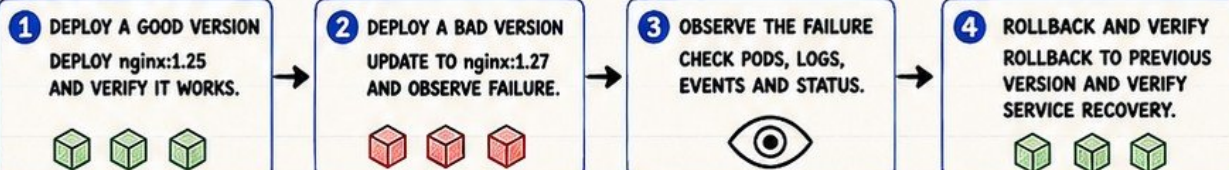


## ROLLBACK

- ✓ RESTORE LAST WORKING VERSION
- ✓ MINIMIZE DOWNTIME
- ✓ MAINTAIN SERVICE AVAILABILITY
- ✓ VERIFY RECOVERY



## PRACTICAL TASK



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



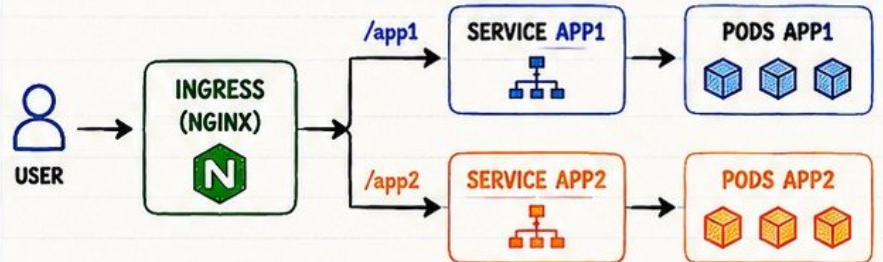
# 13. INGRESS AND APPLICATION ROUTING



## ★ KEY TOPICS

- INSTALL AN INGRESS CONTROLLER
- CREATE INGRESS RESOURCES
- HOST-BASED ROUTING
- PATH-BASED ROUTING
- BACKEND SERVICES
- TEST APPLICATION ROUTES

## INGRESS ARCHITECTURE



## > COMMANDS

- 1 minikube addons enable ingress
- 2 kubectl get pods -n ingress-nginx
- 3 kubectl get ingress
- 4 kubectl describe ingress <ingress-name>
- 5 curl <application-address>

## INGRESS CONTROLLER

INSTALL IN NAMESPACE  
INGRESS-NGINX



## EXAMPLE INGRESS (PATH-BASED ROUTING)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: app-routing
 annotations:
 nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
 rules:
 - host: demo.local
 http:
 paths:
 - path: /app1(/|$)(.*)
 pathType: Prefix
 backend:
 service:
 name: app1-svc
 port:
 number: 80
 - path: /app2(/|$)(.*)
 pathType: Prefix
 backend:
 service:
 name: app2-svc
 port:
 number: 80
```

**PATH-BASED ROUTING**

TRAFFIC IS ROUTED  
BASED ON THE  
URL PATH.

/app1 → APP1  
/app2 → APP2

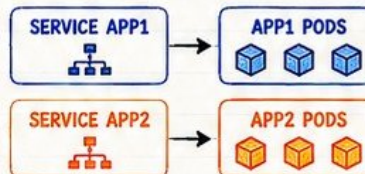
## TESTING APPLICATION ROUTES



ACCESS VIA  
demo.local

curl http://demo.local/app1

curl http://demo.local/app2



## HOST-BASED ROUTING EXAMPLE

app1.demo.local  
→ APP1

app2.demo.local  
→ APP2



## PRACTICAL TASK

DEPLOY TWO APPLICATIONS (APP1 AND APP2) AND ROUTE /APP1 AND /APP2 TO DIFFERENT SERVICES USING AN INGRESS RESOURCE.

- ✓ DEPLOY APP1 AND APP2
- ✓ EXPOSE WITH SERVICES
- ✓ CREATE INGRESS RESOURCE
- ✓ TEST ROUTES



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



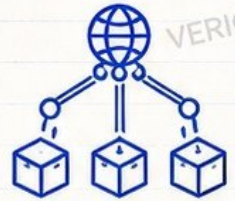
INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



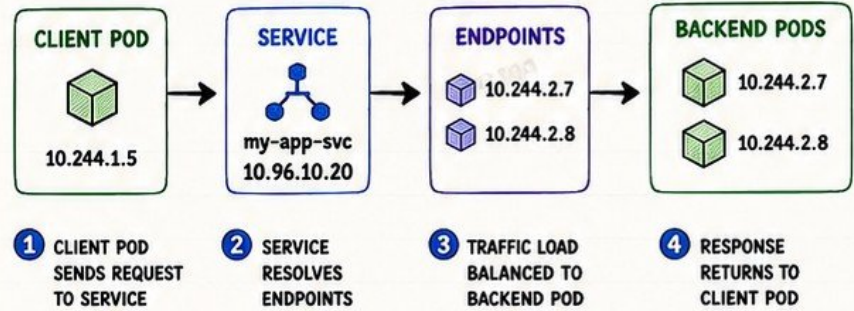
# 14. KUBERNETES NETWORKING LAB



## ★ KEY TOPICS

- POD-TO-POD COMMUNICATION
- POD-TO-SERVICE COMMUNICATION
- SERVICE DNS
- COREDNS
- CLUSTER IPS
- ENDPOINTS
- NETWORK TROUBLESHOOTING

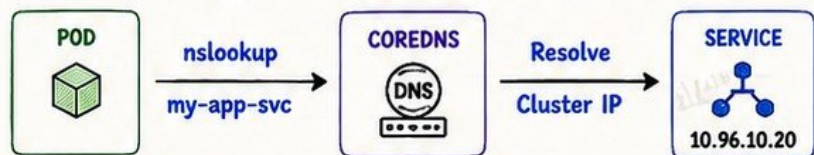
## POD-TO-SERVICE COMMUNICATION FLOW



## > COMMANDS

- 1 `kubectl get pods -o wide`
- 2 `kubectl get svc`
- 3 `kubectl get endpoints`
- 4 `kubectl exec -it <pod> -- sh`
- 5 `nslookup <service-name>`
- 6 `curl <service-name>`

## DNS RESOLUTION WITH COREDNS



- ALL SERVICES GET A STABLE CLUSTER IP
- COREDNS RESOLVES SERVICE NAMES INSIDE THE CLUSTER
- PODS CAN REACH SERVICES USING DNS NAMES

## SAMPLE OUTPUTS

| kubectl get pods -o wide |            |       | kubectl get svc |           |             | kubectl get endpoints |                              |
|--------------------------|------------|-------|-----------------|-----------|-------------|-----------------------|------------------------------|
| NAME                     | IP         | NODE  | NAME            | TYPE      | CLUSTER-IP  | NAME                  | ENDPOINTS                    |
| client-pod               | 10.244.1.5 | node1 | my-app-svc      | ClusterIP | 10.96.10.20 | my-app-svc            | 10.244.2.7:80, 10.244.2.8:80 |
| app-pod-1                | 10.244.2.7 | node1 | ...             | ...       | ...         | ...                   | ...                          |
| app-pod-2                | 10.244.2.8 | node2 |                 |           |             |                       |                              |
| ...                      | ...        | ...   |                 |           |             |                       |                              |

## CLUSTER NETWORKING BASICS

- ✓ EACH POD GETS A UNIQUE IP
- ✓ PODS CAN COMMUNICATE DIRECTLY
- ✓ SERVICES PROVIDE STABLE ACCESS
- ✓ DNS NAMES RESOLVE TO CLUSTER IP
- ✓ ENDPOINTS POINT TO BACKEND POD IPs



## PRACTICAL TASK

TRACE AN HTTP REQUEST FROM ONE POD THROUGH A SERVICE TO ANOTHER POD.

- 1 DEPLOY CLIENT POD AND TWO BACKEND PODS
- 2 CREATE A SERVICE IN FRONT OF BACKEND PODS
- 3 FROM CLIENT POD, CURL THE SERVICE NAME
- 4 OBSERVE REQUEST REACHING BACKEND POD
- 5 VERIFY RESPONSE RETURNS TO CLIENT POD
- 6 CLEAN UP RESOURCES



## NETWORK TROUBLESHOOTING CHECKLIST

- ✓ POD IS RUNNING
- ✓ SERVICE EXISTS
- ✓ SERVICE HAS ENDPOINTS
- ✓ DNS RESOLUTION WORKS

### CHECK WITH:

- `kubectl get pods`
- `kubectl get svc`
- `kubectl get endpoints`
- `nslookup <service-name>`
- `curl <service-name>`

- ✓ NETWORK POLICIES (IF APPLIED)
- ✓ FIREWALL / SECURITY GROUPS
- ✓ CORRECT PORTS
- ✓ APPLICATION LISTENING



## REMEMBER

SERVICES ARE FOR DISCOVERY.  
ENDPOINTS ARE FOR DELIVERY.



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



# 15. RBAC AND SERVICE ACCOUNTS

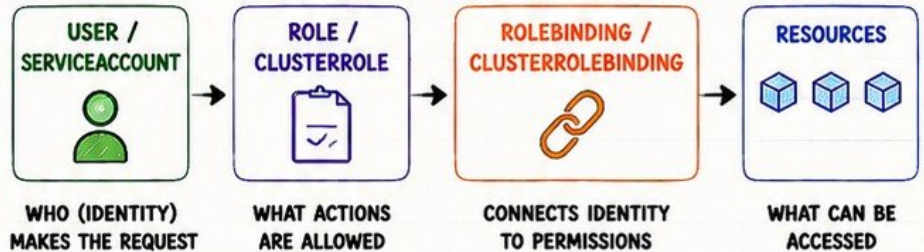


LEAST  
PRIVILEGE  
ACCESS  
ONLY WHAT  
IS NEEDED

## ★ KEY TOPICS

- CREATE SERVICEACCOUNTS
- ROLES
- CLUSTERROLES
- ROLEBINDINGS
- LEAST PRIVILEGE
- PERMISSION TESTING

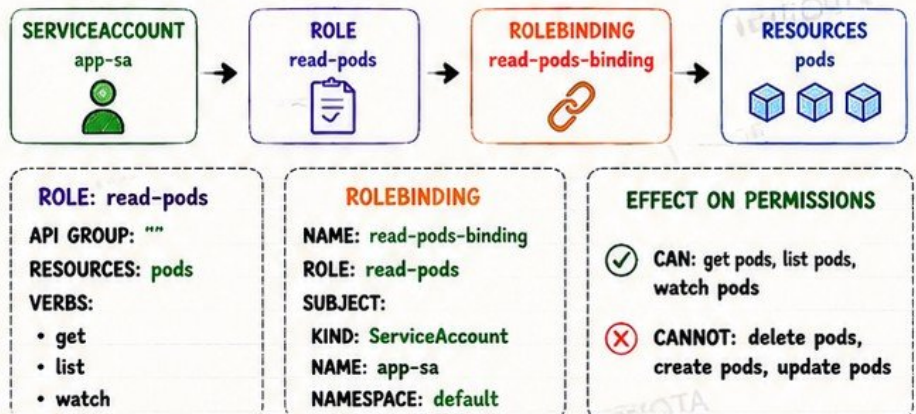
## RBAC BUILDING BLOCKS



## > COMMANDS

- 1 `kubectl create serviceaccount app-sa`
- 2 `kubectl get serviceaccounts`
- 3 `kubectl get roles`
- 4 `kubectl get rolebindings`
- 5 `kubectl auth can-i get pods`
- 6 `kubectl auth can-i delete pods --as=system:serviceaccount:default:app-sa`

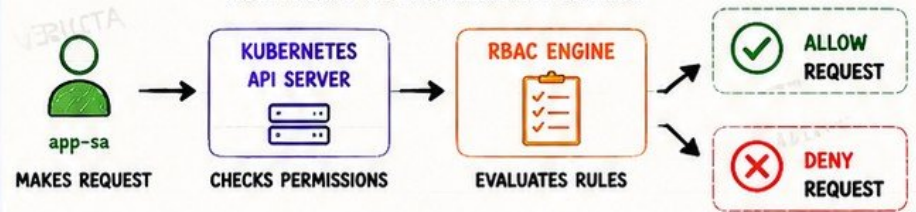
## EXAMPLE: SERVICEACCOUNT WITH READ-ONLY ACCESS TO PODS



## PERMISSION TESTING

- ✓ **ALLOWED**  
\$ `kubectl auth can-i get pods`  
yes
- ✗ **NOT ALLOWED**  
\$ `kubectl auth can-i delete pods --as=system:serviceaccount:default:app-sa`  
no

## HOW RBAC AUTHORIZATION WORKS



## PRACTICAL TASK

GIVE A SERVICEACCOUNT PERMISSION TO READ PODS BUT PREVENT IT FROM DELETING THEM.

- 1 CREATE A SERVICEACCOUNT
- 2 CREATE A ROLE THAT ALLOWS GET, LIST, WATCH ON PODS
- 3 CREATE A ROLEBINDING TO BIND THE ROLE TO THE SERVICEACCOUNT
- 4 TEST: VERIFY IT CAN READ PODS
- 5 TEST: VERIFY IT CANNOT DELETE PODS
- 6 VERIFY LEAST PRIVILEGE ACCESS IS ENFORCED



**BEST PRACTICE: ALWAYS GRANT THE MINIMUM PERMISSIONS REQUIRED. REVIEW ROLES REGULARLY.**



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



# 16. PRACTICAL KUBERNETES TROUBLESHOOTING



## ★ OVERVIEW

Break workloads intentionally to understand how Kubernetes behaves when things go wrong. Learn to investigate the root cause using the right commands and evidence before making changes.

## COMMON FAILURES (BREAK THESE INTENTIONALLY)

### 1. CRASHLOOPBACKOFF



### 2. IMAGEPULLBACKOFF



### 3. PENDING



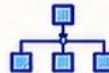
### 4. OOMKILLED



### 5. FAILED PROBES



### 6. INCORRECT SERVICE SELECTOR



### 7. MISSING CONFIGMAP



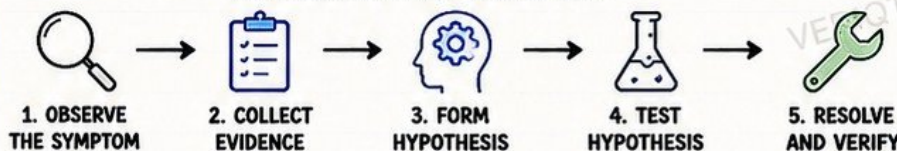
### 8. MISSING SECRET



## > TROUBLESHOOTING COMMANDS

1. `kubectl get pods`
2. `kubectl describe pod <pod>`
3. `kubectl logs <pod>`
4. `kubectl logs <pod> --previous`
5. `kubectl get events`
6. `kubectl get endpoints`
7. `kubectl top pod <pod>`

## TROUBLESHOOTING WORKFLOW



### GOLDEN RULE: DO NOT DELETE THE POD IMMEDIATELY

- COLLECT EVIDENCE FIRST
- UNDERSTAND THE ROOT CAUSE
- THEN APPLY THE RIGHT FIX

## FAILURE CHEAT SHEET

| FAILURE                    | WHAT IT MEANS                        | WHAT TO CHECK                           | KEY COMMANDS                                                                                    | COMMON CAUSES                               |
|----------------------------|--------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------|
| CrashLoopBackOff           | CONTAINER CRASHES REPEATEDLY         | LOGS, CONFIG, CODE, HEALTH CHECKS       | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl logs &lt;pod&gt;</code>          | APP ERROR, BAD CONFIG, FAILING PROBE        |
| ImagePullBackOff           | IMAGE CANNOT BE PULLED               | IMAGE NAME, REGISTRY, IMAGE PULL SECRET | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl get events</code>                | WRONG IMAGE, AUTH ISSUE, PRIVATE REGISTRY   |
| Pending                    | POD CANNOT BE SCHEDULED              | RESOURCES, NODES, AFFINITY, TAINTS, PVC | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl get events</code>                | INSUFFICIENT CPU/MEM, NO NODES, PVC BINDING |
| OOMKilled                  | CONTAINER KILLED DUE TO MEMORY LIMIT | RESOURCE LIMITS, MEMORY USAGE           | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl top pod &lt;pod&gt;</code>       | LOW MEMORY LIMIT, MEMORY LEAK               |
| Failed Probes              | READINESS OR LIVENESS PROBE FAILED   | PROBE CONFIG, APP HEALTH, ENDPOINTS     | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl logs &lt;pod&gt;</code>          | BAD PROBE CONFIG, APP NOT READY             |
| Incorrect Service Selector | SERVICE NOT MATCHING ANY POD         | SERVICE SELECTOR LABELS, POD LABELS     | <code>kubectl describe svc &lt;svc&gt;</code><br><code>kubectl get endpoints &lt;svc&gt;</code> | MISMATCHED LABELS, TYPOS                    |
| Missing ConfigMap          | CONFIGURATION NOT FOUND              | CONFIGMAP NAME, NAMESPACE, MOUNT PATH   | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl get configmap</code>             | CONFIGMAP DOES NOT EXIST OR WRONG NAME      |
| Missing Secret             | SECRET NOT FOUND                     | SECRET NAME, NAMESPACE, MOUNT OR ENV    | <code>kubectl describe pod &lt;pod&gt;</code><br><code>kubectl get secret</code>                | SECRET DOES NOT EXIST OR WRONG NAME         |

## USEFUL CHECKS

- ☒ CHECK POD STATUS
- ☒ READ EVENTS
- ☒ INSPECT POD DETAILS
- ☒ CHECK CONTAINER LOGS
- ☒ CHECK PREVIOUS LOGS
- ☒ VERIFY ENDPOINTS
- ☒ CHECK RESOURCE USAGE
- ☒ VERIFY CONFIGURATION
- ☒ VERIFY SECRETS
- ☒ VERIFY SERVICE ROUTING
- ☒ TEST CONNECTIVITY
- ☒ FORM & TEST HYPOTHESIS



## REMEMBER

EVIDENCE FIRST, ACTIONS SECOND. UNDERSTAND BEFORE YOU CHANGE.



## PRACTICAL TASK

DIAGNOSE EACH FAILURE ABOVE WITHOUT IMMEDIATELY DELETING THE POD.

1. BREAK THE WORKLOAD INTENTIONALLY
2. OBSERVE AND COLLECT EVIDENCE
3. IDENTIFY THE ROOT CAUSE

4. FIX THE ISSUE AND VERIFY
5. DOCUMENT WHAT YOU LEARNED



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



# 17. DEBUG A BROKEN KUBERNETES APPLICATION



## SCENARIO

Users report that the application is unavailable.



## INVESTIGATE EVERY LAYER



DEPLOYMENT



PODS



CONTAINER LOGS



EVENTS



SERVICE



ENDPOINTS



DNS



CONFIGMAP



SECRET



INGRESS



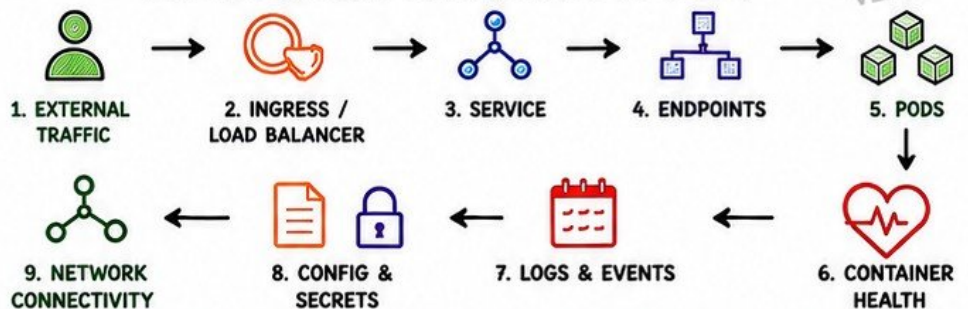
NETWORK CONNECTIVITY



## COMMANDS

1. `kubectl get all`
2. `kubectl describe <resource>`
3. `kubectl logs <pod>`
4. `kubectl exec -it <pod> -- sh`
5. `kubectl get events`
6. `kubectl get endpoints`

## TROUBLESHOOTING PATH (OUTSIDE TO INSIDE)



- MOVE STEP BY STEP
- VERIFY EACH LAYER BEFORE MOVING FORWARD
- ISOLATE THE FAILURE POINT

## WHAT TO CHECK AT EACH LAYER

| LAYER                      | WHAT TO CHECK                                   |
|----------------------------|-------------------------------------------------|
| 1. EXTERNAL TRAFFIC        | CAN USERS REACH THE INGRESS/LOAD BALANCER?      |
| 2. INGRESS / LOAD BALANCER | IS THE REQUEST ROUTED TO THE SERVICE?           |
| 3. SERVICE                 | IS THE SERVICE SELECTOR CORRECT?                |
| 4. ENDPOINTS               | ARE THERE READY ENDPOINTS BEHIND THE SERVICE?   |
| 5. PODS                    | ARE PODS RUNNING AND READY?                     |
| 6. CONTAINER HEALTH        | ARE PROBES PASSING? ANY CRASHES?                |
| 7. LOGS & EVENTS           | WHAT DO LOGS AND EVENTS SAY?                    |
| 8. CONFIG & SECRETS        | ARE CONFIGMAPS AND SECRETS PRESENT AND CORRECT? |
| 9. NETWORK CONNECTIVITY    | CAN PODS REACH EACH OTHER AND THE SERVICE?      |

## QUICK CHECKLIST

- ☒ VERIFY DEPLOYMENT STATUS
- ☒ INSPECT POD STATUS
- ☒ CHECK CONTAINER LOGS
- ☒ REVIEW EVENTS
- ☒ VERIFY SERVICE CONFIGURATION
- ☒ CHECK ENDPOINTS
- ☒ TEST DNS RESOLUTION
- ☒ VERIFY CONFIGMAPS
- ☒ VERIFY SECRETS
- ☒ CHECK INGRESS RULES
- ☒ TEST NETWORK CONNECTIVITY



## PRACTICAL TASK

FOLLOW A STRUCTURED TROUBLESHOOTING PATH FROM EXTERNAL TRAFFIC TO THE APPLICATION CONTAINER AND IDENTIFY THE ACTUAL FAILURE.

1. REPRODUCE THE FAILURE
2. ISOLATE THE ROOT CAUSE
3. DOCUMENT YOUR FINDINGS



## REMEMBER

THE REAL PROBLEM IS RARELY WHERE YOU EXPECT IT! FOLLOW THE PATH. TRUST THE EVIDENCE. NOT ASSUMPTIONS.



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta



# 18. PRODUCTION DEPLOYMENT PRACTICES

## BUILD A PRODUCTION-STYLE WORKLOAD USING:

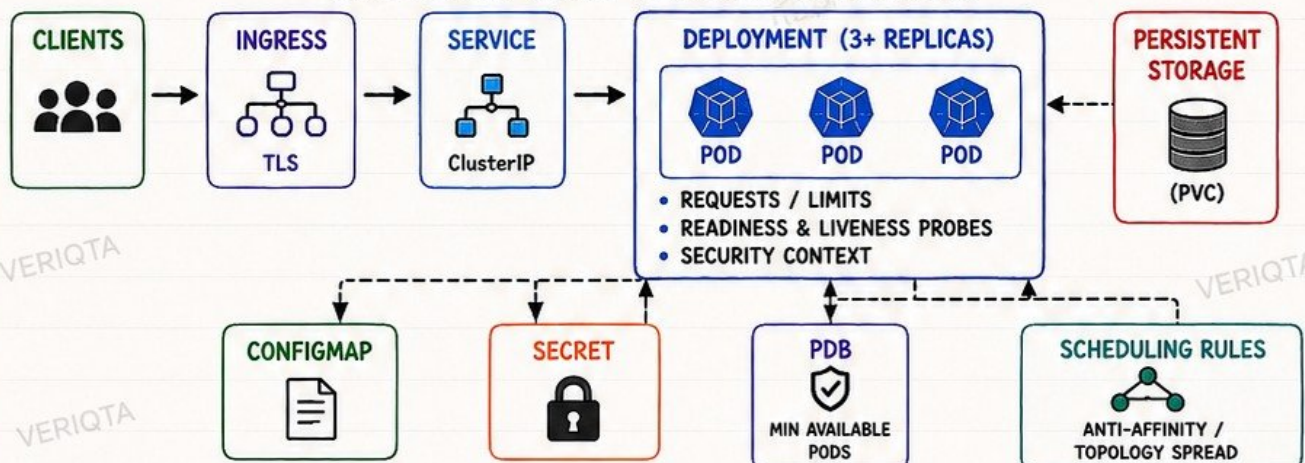
- |                                                       |                                                                          |
|-------------------------------------------------------|--------------------------------------------------------------------------|
| <input checked="" type="checkbox"/> MULTIPLE REPLICAS | <input checked="" type="checkbox"/> SECRETS                              |
| <input checked="" type="checkbox"/> RESOURCE REQUESTS | <input checked="" type="checkbox"/> PERSISTENT STORAGE                   |
| <input checked="" type="checkbox"/> RESOURCE LIMITS   | <input checked="" type="checkbox"/> ROLLING UPDATES                      |
| <input checked="" type="checkbox"/> READINESS PROBES  | <input checked="" type="checkbox"/> POD DISRUPTION BUDGET                |
| <input checked="" type="checkbox"/> LIVENESS PROBES   | <input checked="" type="checkbox"/> POD ANTI-AFFINITY OR TOPOLOGY SPREAD |
| <input checked="" type="checkbox"/> CONFIGMAPS        | <input checked="" type="checkbox"/> SECURITY CONTEXT                     |



## PRACTICAL TASK

Convert a basic Deployment into a production-ready Kubernetes workload.

## PRODUCTION-READY ARCHITECTURE EXAMPLE



## EXAMPLE MANIFEST SNIPPETS

### 1. RESOURCES

```
resources:
 requests:
 cpu: "250m"
 memory: "256Mi"
 limits:
 cpu: "1000m"
 memory: "1Gi"
```

### 2. PROBES

```
readinessProbe:
 httpGet:
 path: /healthz
 port: 8080
 initialDelaySeconds: 10
 periodSeconds: 10
livenessProbe:
 httpGet:
 path: /livez
 port: 8080
 initialDelaySeconds: 20
 periodSeconds: 20
```

### 3. SECURITY CONTEXT

```
securityContext:
 runAsUser: 1000
 runAsGroup: 1000
 readOnlyRootFilesystem: true
 allowPrivilegeEscalation: false
 capabilities:
 drop: ["ALL"]
```

### 5. ANTI-AFFINITY

```
affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values: [web]
 topologyKey: kubernetes.io/hostname
```

### 4. PDB

```
minAvailable: 2
selector:
 matchLabels:
 app: web
```

### 6. ROLLING UPDATE

```
strategy:
 type: RollingUpdate
 rollingUpdate:
 maxUnavailable: 1
 maxSurge: 1
```



## PRODUCTION CHECKLIST

- |                                                               |                                                                 |                                                                             |
|---------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------|
| <input checked="" type="checkbox"/> MULTIPLE REPLICAS RUNNING | <input checked="" type="checkbox"/> CONFIGMAP MOUNTED / USED    | <input checked="" type="checkbox"/> PDB ACTIVE                              |
| <input checked="" type="checkbox"/> REQUESTS AND LIMITS SET   | <input checked="" type="checkbox"/> SECRETS MOUNTED / USED      | <input checked="" type="checkbox"/> ANTI-AFFINITY / TOPOLOGY SPREAD APPLIED |
| <input checked="" type="checkbox"/> READINESS PROBE WORKING   | <input checked="" type="checkbox"/> PERSISTENT STORAGE ATTACHED | <input checked="" type="checkbox"/> SECURITY CONTEXT ENFORCED               |
| <input checked="" type="checkbox"/> LIVENESS PROBE WORKING    | <input checked="" type="checkbox"/> ROLLING UPDATE TESTED       | <input checked="" type="checkbox"/> APPLICATION HEALTHY                     |



YOUTUBE  
@veriqta



GITHUB  
@veriqta



LINKEDIN  
@veriqta



X  
@veriqta



INSTAGRAM  
@veriqta



TELEGRAM  
@veriqta

# PRACTICAL KUBERNETES HANDBOOK

## 19: FAILURE SIMULATION AND RECOVERY LAB



### OBJECTIVE

Intentionally break the application in multiple ways, observe how Kubernetes behaves, and restore each failure.



### SIMULATE THE FOLLOWING FAILURES

- 1 Delete a Pod 
- 2 Kill an application process 
- 3 Break a health check 
- 4 Deploy an invalid image 
- 5 Exhaust memory 
- 6 Break a Service selector 
- 7 Remove configuration 
- 8 Scale the application down 

### RESTORE EACH FAILURE



### OBSERVE USING THESE COMMANDS



`kubectl get pods -w`



`kubectl get events`



`kubectl logs <pod-name>`



`kubectl describe <resource>`



`kubectl top pods`  
`kubectl top nodes`



### WHAT TO OBSERVE

- How Kubernetes detects the failure
- What events are generated
- How Pods are restarted or replaced
- How Services react to endpoint changes
- How Deployments recover desired state
- What happens to availability
- How long recovery takes

### EXAMPLE FAILURE → EXPECTED BEHAVIOR → RECOVERY ACTION

| FAILURE                                                                                                      | EXPECTED BEHAVIOR                            | RECOVERY ACTION                     |
|--------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------------------------------|
|  DELETE A POD             | NEW POD CREATED BY REPLICASET                | VERIFY POD IS RUNNING               |
|  KILL APPLICATION PROCESS | POD RESTARTS (LIVENESS PROBE)                | VERIFY POD IS RUNNING               |
|  BREAK HEALTH CHECK       | POD RESTARTS OR REMOVED (READINESS/LIVENESS) | FIX HEALTH CHECK, VERIFY RECOVERY   |
|  DEPLOY INVALID IMAGE     | POD IN IMAGEPULLBACKOFF OR CRASHLOOPBACKOFF  | DEPLOY VALID IMAGE, VERIFY RECOVERY |
|  EXHAUST MEMORY           | POD OOMKILLED AND RESTARTED                  | INCREASE LIMITS OR OPTIMIZE APP     |
|  BREAK SERVICE SELECTOR   | ENDPOINTS EMPTY, SERVICE DOWN                | FIX SELECTOR, VERIFY ENDPOINTS      |
|  REMOVE CONFIGURATION     | POD MAY FAIL TO START OR CRASH               | RESTORE CONFIG, VERIFY RECOVERY     |
|  SCALE APPLICATION DOWN   | FEWER PODS, REDUCED CAPACITY                 | SCALE BACK UP, VERIFY AVAILABILITY  |



### PRACTICAL TASK

DOCUMENT WHAT KUBERNETES REPAIRS AUTOMATICALLY AND WHAT STILL REQUIRES ENGINEER INTERVENTION.

AUTOMATICALLY  
HANDLED BY  
KUBERNETES



REQUIRES ENGINEER  
INTERVENTION



YouTube  
@veriqta



GitHub  
@veriqta



LinkedIn  
@veriqta



X  
@veriqta



Instagram  
@veriqta



Telegram  
@veriqta



# PRACTICAL KUBERNETES HANDBOOK

## 20: FINAL PRACTICAL PROJECT, PRODUCTION-STYLE KUBERNETES APPLICATION

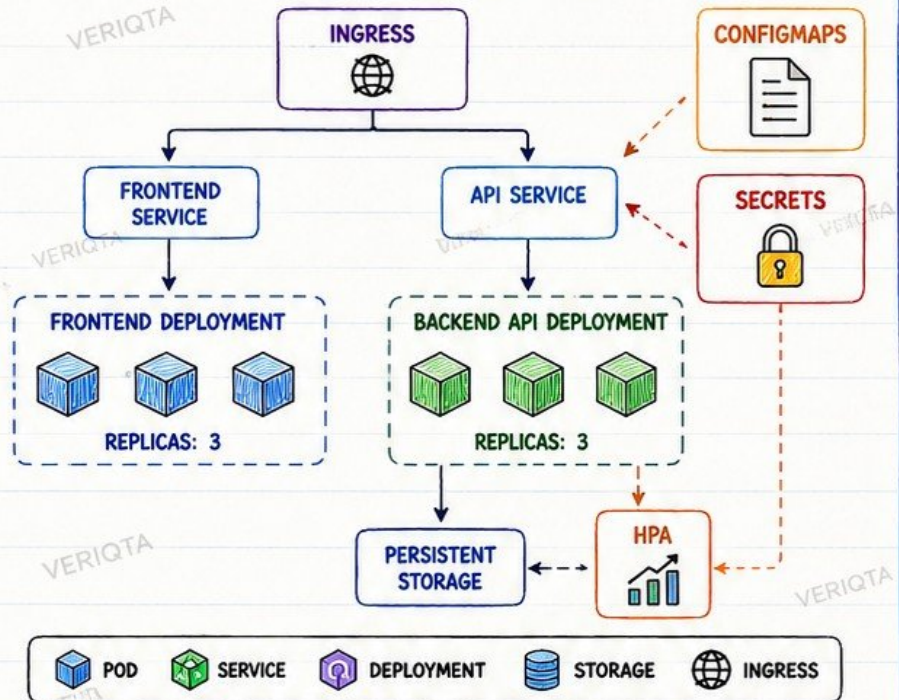


### BUILD AN END-TO-END KUBERNETES ENVIRONMENT

- ✓ NAMESPACE
- ✓ FRONTEND
- ✓ BACKEND API
- ✓ MULTIPLE REPLICAS
- ✓ DEPLOYMENTS
- ✓ SERVICES
- ✓ CONFIGMAPS
- ✓ SECRETS
- ✓ PERSISTENT STORAGE
- ✓ INGRESS
- ✓ RESOURCE REQUESTS AND LIMITS
- ✓ HEALTH PROBES
- ✓ HORIZONTAL POD AUTOSCALER
- ✓ RBAC
- ✓ ROLLING DEPLOYMENT STRATEGY
- ✓ BASIC SECURITY CONFIGURATION



### APPLICATION ARCHITECTURE (EXAMPLE)



### THEN PERFORM THE FOLLOWING

- 1 DEPLOY THE COMPLETE APPLICATION.
- 2 VERIFY POD AND SERVICE CONNECTIVITY.
- 3 TEST APPLICATION ROUTING.
- 4 GENERATE TRAFFIC.
- 5 OBSERVE RESOURCE UTILIZATION.
- 6 TEST AUTOSCALING.
- 7 DEPLOY A NEW APPLICATION VERSION.
- 8 INTRODUCE A BROKEN RELEASE.
- 9 TROUBLESHOOT THE FAILURE.
- 10 ROLL BACK THE DEPLOYMENT.
- 11 DELETE A POD AND VERIFY SELF-HEALING.
- 12 SIMULATE ANOTHER PRODUCTION FAILURE.
- 13 RECOVER THE APPLICATION.
- 14 VERIFY APPLICATION HEALTH.
- 15 REVIEW EVENTS AND LOGS.
- 16 CLEAN UP EVERY RESOURCE CREATED DURING THE PROJECT.

### USE THESE COMMANDS DURING THE PROJECT

- ✓ `kubectl get pods -w`
- ✓ `kubectl get svc`
- ✓ `kubectl get all`
- ✓ `kubectl get ingress`
- ✓ `kubectl get hpa`
- ✓ `kubectl top pods`
- ✓ `kubectl top nodes`
- ✓ `kubectl describe <resource>`
- ✓ `kubectl logs <pod>`
- ✓ `kubectl get events`
- ✓ `kubectl rollout status <deploy>`
- ✓ `kubectl rollout history <deploy>`
- ✓ `kubectl rollout undo <deploy>`



### FINAL OUTCOME

BY PAGE 20, THE READER SHOULD NOT SIMPLY KNOW KUBERNETES TERMINOLOGY.

THEY SHOULD HAVE PERSONALLY BUILT AND OPERATED A COMPLETE PRODUCTION-STYLE KUBERNETES APPLICATION FROM START TO FINISH.



### YOU SHOULD HAVE PERSONALLY:

- ✓ BUILT A CLUSTER
- ✓ WRITTEN KUBERNETES YAML
- ✓ DEPLOYED APPLICATIONS
- ✓ EXPOSED SERVICES
- ✓ CONFIGURED APPLICATIONS
- ✓ MANAGED SECRETS
- ✓ USED PERSISTENT STORAGE
- ✓ CONFIGURED HEALTH CHECKS
- ✓ SET CPU AND MEMORY CONTROLS
- ✓ SCALED WORKLOADS
- ✓ PERFORMED ROLLING DEPLOYMENTS
- ✓ ROLLED BACK FAILURES
- ✓ CONFIGURED INGRESS
- ✓ WORKED WITH RBAC
- ✓ TROUBLESHOOT BROKEN WORKLOADS
- ✓ SIMULATED PRODUCTION FAILURES
- ✓ RECOVERED APPLICATIONS
- ✓ BUILT A COMPLETE PRODUCTION-STYLE KUBERNETES DEPLOYMENT

